

SmartHire — Resume-to-Job Matching & Career Guidance Engine

A complete, end-to-end machine learning project combining supervised and unsupervised techniques to recommend jobs, predict shortlisting, and generate CV improvement reports.

Supervised + Unsupervised ML

Python · scikit-learn · Streamlit

Duration: 3 weeks

Team size: 2–3 or Individual

1 Project Overview

You will build **SmartHire**, a portal where a student uploads a resume (CV) and the system returns: (a) a ranked list of matching jobs from a job database, (b) a fit/shortlisting score for target roles, and (c) a skill-gap report telling the candidate what to improve.

The entire system must be built using **classical machine learning** — no large language models or generative AI. You will use real-world public datasets, engineer features, train and evaluate models, and deliver a working web demo.

About "searching LinkedIn / Naukri live"

Live scraping of LinkedIn, Naukri, or similar sites **is not allowed** — it violates their Terms of Service and is not a machine learning task. Instead, you will work with **publicly available job datasets** (downloaded once), which is exactly how industry teams prototype recommendation systems. Your "search" becomes matching the CV against this job database using ML.

2 Datasets to Use

Download these from Kaggle (free account needed). Place them under `data/raw/`.

- **Resume Dataset** — ~960 resumes labeled into 25 job categories (for the classifier).
- **Naukri Job Listings** — Indian job postings: title, skills, experience, salary.
- **LinkedIn Job Postings 2023–2024** — large set of detailed job descriptions.
- **Indeed / Job Recommendation datasets** — optional, for extra job variety.

Building the job corpus

Merge the job datasets into one clean job corpus (common columns: *title, company, location, skills, description, experience*). This single corpus is what the recommender searches against.

3 Requirements → Machine Learning Tasks

Here is how the original portal idea maps to concrete ML problems:

Portal feature	ML task	Type
Search & shortlist related jobs	Content-based job recommendation (CV ↔ job similarity ranking)	Unsupervised
Give a proper ranked job list	Top-N ranking by match score	Unsupervised
CV preparation help	Resume category classification + skill-gap detection	Supervised
"Is my CV good for this role?"	Shortlisting / fit prediction	Supervised
Discover job families / role groups	Job clustering & topic modeling	Unsupervised
"Mentor" doubt-clearing (optional)	Rule-based answers driven by ML outputs (not an LLM)	Stretch goal

About the "AI chat mentor"

A conversational AI mentor is a generative-AI feature, not classical ML, so it is **out of scope** for the core project. If you want a mentor feature, build a simple **rule-based assistant** that answers questions like "What skills am I missing for Data Analyst?" using your skill-gap module's output.

4 Supervised Learning Components

MODEL A

Resume Category Classifier

Input: resume text → **Output:** job domain (e.g., Data Science, Web Dev, HR).

Pipeline: clean text → TF-IDF → Logistic Regression / Random Forest / SVM.

Learn: text vectorization, multi-class classification, confusion matrix.

MODEL B · OPTIONAL

Shortlisting / Fit Predictor

Input: features from (resume + a specific job) → **Output:** fit (0/1) or score.

Features: skill overlap, experience match, education match, text similarity.

Models: Logistic Regression → XGBoost; compare results.

Learn: feature engineering, class imbalance, precision/recall/ROC-AUC.

OPTIONAL

Salary Band Predictor

A simple regression on job features to estimate salary range. Good practice for regression metrics (MAE, RMSE).

5 Unsupervised Learning Components

CORE ENGINE

Job Recommendation

Vectorize all job descriptions and the resume (TF-IDF), compute **cosine similarity**, return the top-N matching jobs. This is the heart of the portal.

Learn: vector space model, similarity, content-based recommenders.

DISCOVERY

Job / Role Clustering

K-Means or Hierarchical clustering on job vectors to find natural job families. Choose k using the elbow method + silhouette score.

Learn: clustering, PCA / t-SNE for visualization.

INSIGHT

Skill-Gap Report

Compare the candidate's skills against the target cluster's top skills to produce a **skill-gap report** (this becomes the "CV preparation" feature).

Optional: use LDA / NMF topic modeling on job descriptions to extract in-demand skill themes for richer gap analysis.

6 System Architecture

```
Resume upload (PDF / DOCX / text)
  |
  v
Text extraction → Preprocessing (clean, tokenize, vectorize)
  |
  ├──> Classifier (supervised)      → predicts target role
  ├──> Similarity engine (unsup.)  → top-N matching jobs + scores
  ├──> Fit predictor (supervised)  → shortlisting probability
  └──> Skill-gap module (unsup.)   → CV improvement report
  |
  v
Streamlit / Flask web portal
(shows recommended jobs, fit scores, and skill gaps)
```

7 Project Directory Structure

Organize your repository exactly like this. Keep notebooks for exploration and `src/` for reusable code.

```
smarthire/
├── README.md                # project intro, setup, how to run
├── requirements.txt         # pinned dependencies
├── .gitignore              # ignore data/, venv/, *.pkl
├── data/
│   ├── raw/               # original downloaded datasets (never edit)
│   ├── interim/           # merged / partially cleaned data
│   └── processed/         # final model-ready data
├── notebooks/
│   ├── 01_eda.ipynb        # explore jobs + resumes
│   ├── 02_resume_classifier.ipynb # supervised: category model
│   ├── 03_recommender.ipynb # unsupervised: similarity ranking
│   ├── 04_clustering_topics.ipynb # unsupervised: clusters + topics
│   └── 05_fit_predictor.ipynb # supervised: shortlisting model
├── src/
│   ├── __init__.py
│   ├── config.py          # paths, constants, params
│   ├── data/
│   │   ├── load_data.py   # read raw datasets
│   │   └── preprocess.py  # clean text, merge job corpus
│   ├── features/
│   │   ├── text_features.py # TF-IDF / vectorizers
│   │   └── match_features.py # skill overlap, experience match
│   ├── models/
│   │   ├── classifier.py   # train/predict resume category
│   │   ├── recommender.py  # cosine-similarity job ranking
│   │   ├── clustering.py   # KMeans + topic modeling
│   │   └── fit_predictor.py # shortlisting model
│   ├── parsing/
│   │   └── resume_parser.py # extract text from PDF/DOCX
│   └── evaluate.py        # metrics for all models
├── models/                # saved .pkl model files
│   ├── classifier.pkl
│   ├── tfidf_vectorizer.pkl
│   └── fit_predictor.pkl
├── app/
│   └── streamlit_app.py    # the web portal UI
├── reports/
│   ├── figures/           # PCA/t-SNE plots, confusion matrices
│   └── final_report.pdf   # written project report
└── tests/
    └── test_features.py    # basic unit tests (optional)
```

8 Milestones & Timeline

- Week 1** Data collection & cleaning; EDA; build the merged job corpus; resume category classifier (supervised) + evaluation.
-
- Week 2** Similarity-based job recommender (unsupervised) with top-N output; clustering + skill-gap report. (*Topic modeling is optional.*)
-
- Week 3** Integrate all modules into the Streamlit portal; write report and prepare presentation. (*Shortlisting / fit predictor is optional — add it only if time allows.*)
-

Minimum scope to pass

The **resume classifier + job recommender + skill-gap report** alone make a complete supervised-plus-unsupervised project. The **fit/shortlisting predictor** and **topic modeling** are **optional** — attempt them only after the core is working.

9 Evaluation Metrics

- **Classification (classifier & fit predictor):** accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix.
- **Clustering:** silhouette score, inertia (elbow method).
- **Recommender:** Precision@K, plus a qualitative check that the top jobs make sense for sample resumes.
- **Regression (optional salary model):** MAE, RMSE, R^2 .

10 Deliverables

1. Cleaned datasets + EDA notebook.
2. The resume classifier (supervised) with metrics; the fit predictor (optional) with a comparison table if attempted.
3. Recommender + clustering notebooks with visualizations (PCA/t-SNE, silhouette plots).
4. A working Streamlit demo: upload a CV → see matched jobs, fit scores, and skill gaps.
5. A written report explaining your data, model choices, results, and limitations.
6. The full Git repository following the structure in Section 7.

11 Stretch Goals (for stronger teams)

- Replace TF-IDF with sentence embeddings (still ML — improves match quality).
- Add a learning-to-rank model for ordering job results.
- Build a rule-based "mentor" that answers skill questions from the skill-gap output.
- Deploy the Streamlit app online (Streamlit Community Cloud) and share a public link.